Demo Provenance-Aware Question Answering Gateway

Contents

Abstract

This document presents a demo of provenance-aware question answering over relational data. The demo lets a user inspect a query plan, choose how each source table is processed, run the resulting pipeline, and trace every answer tuple back to its supporting rows.

1 Motivation and Objectives

Large language models can translate questions, synthesize answers, and manipulate structured representations, but a fluent answer alone does not show which source records support it. For question answering task, a system should output both the answer and the evidence used to derive it.

This demo has the following objectives:

- answer questions over tabular datasets through several alternative pipelines;
- preserve stable row identifiers from source data through retrieval and generation;
- compute or explain provenance using the records that contributed to an answer;
- expose plans, retrieved context, prompts, model output, validation results, generated CSV files, and service responses for inspection;
- provide an interface for comparing base, fine-tuned, planner-based, and internal-knowledge configurations.

2 Demo Use Case

Consider a motorsport analyst exploring the RELF1 dataset who asks which 2009 Formula 1 races took place in Australia and at which circuits:

```
SELECT r.name AS race_name,
       c.name AS circuit_name,
       c.location
FROM races r
JOIN circuits c ON r.circuitId = c.circuitId
WHERE r.year = 2009 AND c.country = 'Australia'
ORDER BY r.date
LIMIT 5;
```

The analyst does not only want the race and circuit names. They also want to verify how the answer was obtained: which race record was joined with which circuit record, how the year and country filters were applied, which execution pipeline handled each table, and which exact source rows support each result.

This scenario demonstrates the full use of the gateway. Before executing the query, it shows a relational plan split into tables leaf tasks. The user can accept the default planner-only pipeline or select a different strategy for each leaf. During execution, the interface reports retrieval and processing progress. For the explanation part, the gateway exports the selected rows as CSV files and invokes the AP Explanation service. This executes the query on ProvSQL, creating a new d postgres database on which the query is run and the provenance is computed, also a written explanation for the provenance. It presents three result views:

- **Answer**: the final tuples returned by the query;
- **Provenance**: the witness sets that explain which row identifiers support each tuple;

- **Supporting rows**: the source records referenced by those identifiers, including their table and values.

3 Step-by-Step UI Walkthrough

After starting the container, open `http://127.0.0.1:9006/ui/`. The root URL redirects to the same interface. The following sequence is suitable for a live demo.

1. **Select a dataset.** In the *Dataset* menu, choose `RELF1` or `TPCH`. Changing the dataset clears any existing plan, so generate a new plan after switching.

2. **Enter the demo query.** Replace the filled query with the SQL query.

3. **Generate the plan.** Select *Generate plan*. In the *Query plan* panel, point out the dataset and query summary, then follow the displayed flow from reading source tables, through the join, ordering, and projection, to the final tuples with provenance.

4. **Inspect the leaf tasks.** Each leaf card shows the scan type, local predicate, join keys, selected columns, and leaf SQL.

5. **Choose a pipeline for each leaf.** Leave `Planner only` for a short baseline run, or select `Planner only + explanation` to include the AP explanation stage. Other choices support comparisons with predicate pushdown, iterative join-aware retrieval, standard RAG, internal knowledge, or manual review. Pipeline choices are made per table, so a single run can combine strategies.

6. **Run the selection.** Select *Run selected pipelines*. The status badge changes from *Waiting* to *Running*, and the Answer view receives progress events for each leaf, retrieved context, CSV generation, answer construction, and explanation. The first run can take longer if a FAISS index must be built.

7. **Read the answer.** When the status becomes *Complete*, use the *Answer* tab to show the answer produced. If CSVs were generated, *View generated CSVs* opens the exact csv files passed to the explanation service. If an AP explanation was requested, *Show AP explanation* shows it.

8. **Trace provenance.** Open the *Provenance* tab. This is the explanation of why the result exists.

9. **Verify the evidence.** Open *Supporting rows* and match the row identifiers from the provenance formula to the original table names and values.

If a run reports an error, retain the visible partial progress and inspect `docker compose logs -f gateway`. This is especially useful during a live demo because it shows a planning, retrieval, model, or explanation service failure.

4 System Architecture

The browser sends planning and execution requests to the FastAPI gateway. The gateway parses the SQL into a query plan, dispatches each leaf task through its selected pipeline, validates row identifiers, combines the leaf results, and returns answer tuples with provenance. Ollama serves the local base and fine-tuned models, while FAISS provides row-level retrieval over the selected dataset.

When explanation is selected, the gateway writes one CSV per contributing table to a shared bucket and submits an explanation task. The AP Explanation service reads those files, uses PostgreSQL with ProvSQL for provenance-aware relational execution, and uses Redis for asynchronous task coordination. Its response is streamed back through the gateway and displayed in the UI.

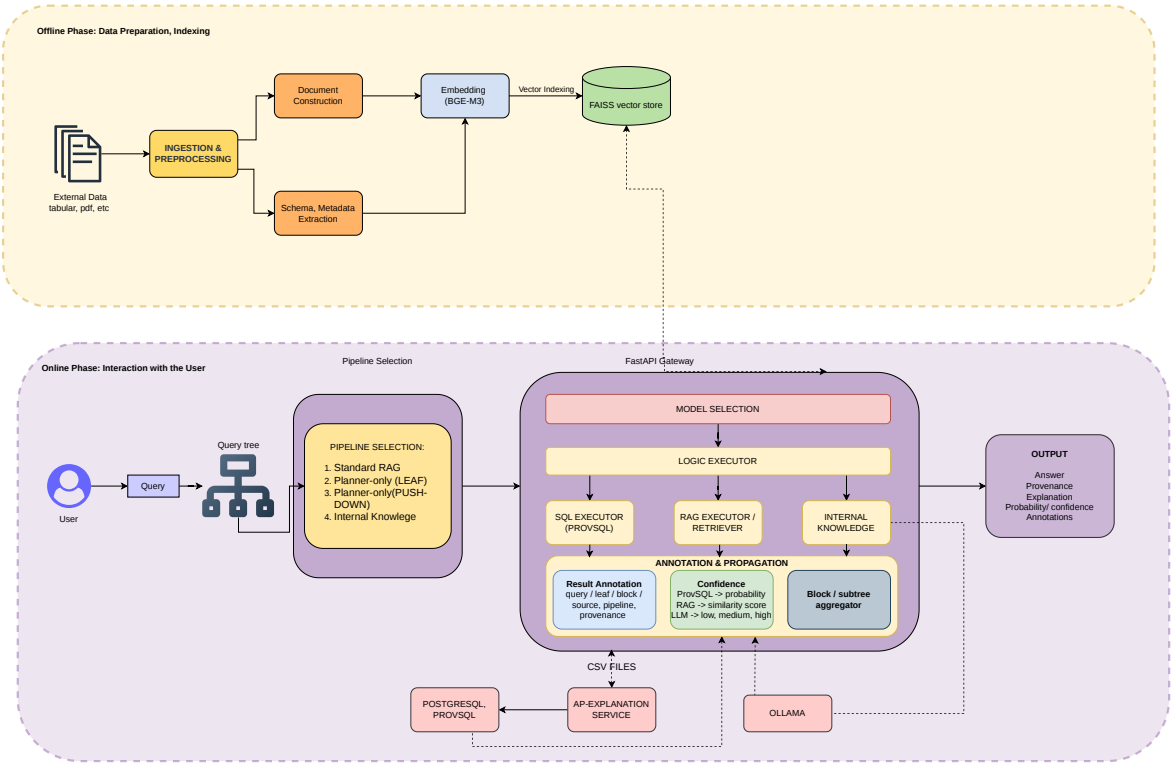

Figure 1: Demo gateway architecture and the main request path.

The key architectural boundary is between orchestration and evidence. The gateway may use different LLM and retrieval strategies to obtain candidate rows, but all successful leaf outputs must satisfy the same validated row-level contract. Stable row identifiers then connect final answers, provenance formulas, supporting-row views, exported CSVs, and the external explanation service.

5 Running the Demo

The demo requires Docker with Compose support, Git, and Git LFS. From the repository root, run:

```
git lfs pull
docker compose up -d --build
docker compose ps
```

Open `http://127.0.0.1:9006/ui/` when the services are ready. Model loading and first-time index creation can delay the first request; their progress is visible with `docker compose logs -f gateway`. The current Compose stack exposes the explanation service on host port `5002` and keeps Ollama on the internal network.

6 Related Work

6.1 Retrieval-Augmented Generation

Retrieval over tabular data requires different design choices from standard document retrieval. A common approach in RAG systems is to linearize tables into text and retrieve them as ordinary chunks. However, recent work has shown that this strategy may lose important tabular structure and provide only a partial view of the data, especially for questions requiring joins, filtering or aggregation. TableRAG [7] addresses this limitation by combining text retrieval with SQL-based execution over tabular components for heterogeneous document reasoning, showing the importance of preserving the distinction between textual and structured evidence. During an offline phase, the system extracts table contents, builds textual chunks for retrieval, stores table schemas, and also ingests the tables into a relational database. During online inference, it decomposes the user question, retrieves relevant chunks, maps retrieved table fragments back to their corresponding schemas, generates SQL when tabular reasoning is required, executes the SQL query over the relational database, and combines the execution result with retrieved textual evidence to generate the answer.

Our retrieval component follows a related motivation, but adapts it to relational CSV data. Instead of indexing arbitrary table chunks, we create one document for each table row. Each document contains a textual representation of the row, including the table name, primary-key values, column-value pairs, and selected foreign-key links to related rows. This row-level representation is designed to preserve enough relational structure for retrieval while remaining compatible with dense vector search. In addition, the document metadata stores the original table, row identifier, primary key, full row values, provenance row number when available, and linked-row information. The resulting documents are embedded with BGE-M3 and indexed with FAISS, enabling the system to retrieve candidate records that can later be used by the LLM.

Our retrieval design is also influenced by schema-aware approaches such as RAT-SQL [6]. RAT-SQL addresses text-to-SQL parsing by making the structure of the database schema explicit: tables and columns are represented as graph nodes, while relations such as primary keys, foreign keys, and column-table membership are encoded as graph edges. This allows the model to reason not only over the textual form of the schema, but also over the relational connections that determine how tables and columns should be interpreted.

Although our system does not adopt RAT-SQL's neural parser, it follows a similar principle at the retrieval level. Each indexed document is not a plain textual chunk, but a structured representation of a relational row. In this way, relational information that would be hidden or lost in a naive text representation is made explicit before embedding and retrieval.

6.2 Data Provenance

Data provenance, by explaining how the result of an operation is derived from its inputs, has proven to be a crucial concept across a wide variety of applications. As highlighted by [3], provenance enables fundamental functionalities such as debugging queries and transformations, assessing the trustworthiness of data, interpreting complex data processing pipelines, and performing incremental maintenance of query results. It also supports explaining unexpected query outcomes and reasoning about hypothetical changes to inputs and outputs.

These are the core provenance models: **Lineage** [1], based on data dependencies that track data dependencies separately for each input relation of a query. Lineage focuses on identifying the specific input tuples that contribute to each output tuple. It provides an efficient way to trace data origin. The lineage is computed by breaking down the query into smaller parts, then computing which tuples in subqueries contributed to each intermediate result, and finally combining all their lineages to get the final answer. **Why provenance** [1], which captures sets of input tuples (witnesses) that are sufficient to produce a particular output tuple. It reflects

the minimal explanations of why an answer exists. **How provenance** [2], which goes further by expressing the precise combination and multiplicity of input tuples used to compute each output. **Where provenance** [1], which focuses on the origin of individual data values. While why-provenance captures the relationship between source and output tuples, where provenance traces the exact source locations (specific rows in the input relations) from which output values are copied. This notion highlights the link between input and output attributes, but, unlike why provenance, it is not invariant under query equivalence. Moreover, the article covers key algorithms for provenance tracking, the computational complexity of various provenance models, and the integration of provenance computation into query engines and data management systems.

One of the key techniques discussed in the article for tracking data provenance is the use of **semirings**, an algebraic framework that provides a unified way to compute and represent provenance information during query processing. A fundamental contribution in this field was achieved through the provenance **semiring framework** [4], which models various forms of provenance, including why-provenance. In this framework, relations are annotated with elements of semirings and relational algebra operators are mapped to semiring operations. Using the semiring framework, each tuple in the database is bound with a provenance annotation. As a query is executed, these annotations are propagated and combined according to the query's structure. What makes the semiring model powerful is that it defines two operations, addition and multiplication, to determine how provenance annotations should be handled at each step of the query. Addition is used when a query operation (like a union) can produce the same output tuple in multiple ways. In this case, the provenance of the output is the combination of the alternative sources. Multiplication is used when the result depends on the combination of multiple input tuples (as in a join). Here, the provenance of the result reflects the joint contribution of those inputs.

Our project is based on the notion of *why-provenance*. Why-provenance is a fundamental provenance model originally introduced by Buneman et al. (2001) [1] to formalize the explanation of query results based on the principles of sufficiency and minimality. In this framework, the provenance of a query result is characterized by identifying subsets of the input database, called *witnesses*, each of which alone suffices to produce that result. **Witness sets** are derived from the semiring expression by enumerating the minimal input tuple combinations that are sufficient to produce the output tuple. Each multiplicative term in the provenance polynomial corresponds to one witness set. Formally, the *set of witnesses* for a tuple t in the output of a query Q over a database D is defined as the collection of all subsets $D' \subseteq D$ such that $t \in Q(D')$. Since this set can contain redundant witnesses that include unnecessary tuples, the notion of *minimal witnesses* is introduced. Minimal witnesses are those subsets that are minimal under set inclusion, meaning they do not contain any smaller witness that also produces t. This minimality ensures the provenance explanations focus on essential and non-redundant parts of the input data. We use this concept to enhance the interpretability of outputs produced by a Retrieval-Augmented Generation (RAG) pipeline. We use the notion of *witness sets*, which act as textual justifications for each item of an answer, effectively mapping the output back to its supporting retrieved content.

6.3 Databases and LLMs

Recent work has investigated how database query-processing ideas can be adapted to interactions with Large Language Models. Satriani et al. introduce Galois [5], a system that executes SQL queries over LLMs by treating the model as a data source and by defining LLM-aware logical and physical operators. In particular, Galois introduces LLMScan and Filter-LLMScan operators, which retrieve tuples from the model either without or with pushed-down selection conditions. Their work shows that complex data-oriented queries can benefit from being decomposed into operator-level plans, and that deciding whether conditions should be pushed into the LLM call involves a trade-off between cost and result quality.

Our demo system takes inspiration from this operator-oriented view, but adapts it to a different setting. In our planner-based pipelines, we introduce analogous LeafScan and FilterLeafScan steps. These steps are applied to the leaves of a generated tree plan and are used to query the LLM on smaller tasks, optionally with filter conditions pushed into the leaf. Unlike Galois, however, our system does not treat the LLM as the primary storage layer. The data source remains an explicit collection of relational tables, and the LLM is used as part of a pipeline involving retrieval, planning, structured generation, deterministic execution, and provenance explanation.

This distinction is central to our use of the idea. While Galois studies how SQL execution can be optimized when the LLM itself is queried as a data source, our system studies how LLM calls can be organized around a planner when answering questions over existing tabular data.

7 Pipeline Modes

7.1 Standard RAG

The model identifiers `base-llama3-8b`, `best-ft-llama3-8b-nl`, and `best-ft-llama3-8b-sql` use the standard path. The gateway retrieves rows using the complete question, serializes the exact source rows into a prompt, and asks the selected model to return the answer and provenance. This provides a useful end-to-end baseline, although reasoning, row selection, and provenance generation remain largely model-controlled.

7.2 Internal-Knowledge Baseline

The `internal-knowledge` mode performs no source-row retrieval. It asks the model to answer from its internal parameters, so its provenance must not be treated as verified source provenance unless it is separately resolved and validated.

7.3 Planner-Only

The `planner-only` mode accepts SQL and decomposes it into table-level leaf tasks. The plan identifies tables, aliases, predicates, join keys, selected and grouping columns, aggregates, ordering, limits, and deterministic post-operations. Each leaf retrieves candidate rows and asks the model to copy relevant rows under a strict output contract. The UI exposes the plan and leaf traces so the user can inspect the retrieval query, context, model output, parsed rows, and validation status.

7.4 Planner-Only with Predicate Pushdown

`planner-only-pushdown` adds the leaf's predicates and required columns to its retrieval query. Exposing values such as year, country, driver, or circuit names to FAISS can improve candidate recall for selective RELF1 questions. This mode is selected per leaf through the UI.

7.5 Iterative Join-Aware Pipeline

`iterative-join-aware` coordinates retrieval across joined tables. It starts with a leaf that has useful predicates or join keys, extracts validated rows, propagates join-key values to connected leaves, and enriches subsequent retrieval queries with those bindings. This makes the evidence discovery process follow the join structure, although its result still depends on the recall of the starting leaf and the correctness of the extracted join edges.

7.6 Planner-Only Explanation

`planner-only-explanation` extends planner-only extraction by writing the validated rows to per-table CSV files and submitting them with the SQL query to the AP Explanation service. The service executes the relational and provenance workflow, and the UI displays its written explanation alongside the answer, provenance formula, and supporting rows.

The modes are deliberately exposed side by side in the demo. A presenter can use one strategy for every leaf or mix them per table to compare a baseline, predicate-aware retrieval, join-guided retrieval, and external provenance explanation under the same query plan.

A Detailed Component Architecture

The deployed system contains the following services.

Component	Responsibility
FastAPI gateway	API routing, dataset loading, retrieval, planning, prompting, validation, logging, UI hosting, and orchestration.
Ollama	Local serving of the base and fine-tuned Llama models.
OpenAI-compatible LLM endpoint	Optional provider for planner leaf calls, selected through configuration.
FAISS	Dense row-level indexes, one index per configured dataset.
AP Explanation service	Loads generated CSV relations, executes the requested query, and returns answer and provenance information.
PostgreSQL with ProvSQL	Relational execution and provenance support for the explanation service.
Redis	Task broker and result backend used by the explanation service.
Shared bucket	File exchange area for generated per-table CSV files.

The gateway and explanation service share `./bucket` through different container mount points. A process-wide lock serializes explanation pipelines because each run cleans and rewrites this shared directory.

A.1 Main Implementation Files

File	Purpose
`gateway.py`	FastAPI application and pipeline orchestration.
`planner.py`	SQL parsing and structured query-plan construction using `sqlglot`.
`faiss_index_manager.py`	Lazy loading, building, checkpointing, and persistence of FAISS indexes.
`embedding_strategies.py`	Embedding provider and device selection.
`prompt.py`	Strict prompts for ordinary and join-aware leaf extraction.
`iterative_join_pipeline.py`	Join-guided ordering, binding propagation, and leaf retrieval.
`explanation_pipeline.py`	CSV generation, explanation-service call, and Markdown rendering.
`explanation_client.py`	Synchronous and asynchronous AP Explanation API client.
`json_to_csv.py`	Conversion of validated leaf rows into one CSV file per table.
`prompt_internal_knowledge.py`	Dataset-specific prompts for the no-retrieval baseline.

File	Purpose
ui/	Browser interface served by the gateway.

B Data Model and Dataset Handling

B.1 Supported Datasets

The gateway currently defines configurations for:

- tpch, loaded from CSV_DIR, with its schema defined in tpch_schema_info.py; and
- rel-f1, loaded from RELF1_CSV_DIR, with schema information derived from schema_profile_relf1.json.

The request field dataset selects a configuration. If it is omitted, DEFAULT_DATASET is used. Dataset names are normalized before lookup; unsupported names are rejected.

B.2 Stable Row Identifiers

Each CSV table is expected to contain a provenance row-number column such as nation_rownum. During loading, this value is exposed internally as __rid__ and used as the stable identifier for retrieval, validation, provenance resolution, and CSV export.

The gateway maintains, separately for each dataset:

- an in-memory cache of all loaded rows grouped by table;
- a table-local map from row identifier to row position;
- a global map from row identifier to the table and row position; and
- lazily initialized embedding and FAISS objects.

Row identifiers must be globally unambiguous within a dataset for provenance resolution to be reliable.

B.3 CSV Loading

CSV files are loaded lazily. The gateway detects delimiters, reads values as text, establishes the row identifier, and constructs its lookup indexes. Because CSV values are textual, the gateway contains dataset-aware SQL rewriting for quoted identifiers and numeric comparisons before sending some queries to the external relational service.

C Row-Level Indexing and Retrieval

C.1 Indexed Representation

FaissIndexManager creates one vector document per row. Its text has the form:

```
table: nation | n_nationkey: 1 | n_name: ARGENTINA | n_regionkey: 1
```

Internal fields such as __rid__ and columns ending in _rownum are excluded from the embedded text. FAISS metadata stores table and rid, allowing the gateway to discard the approximate document representation and reconstruct the exact row from the CSV cache after retrieval.

If a non-empty index directory exists, it is loaded. Otherwise, the index is built in batches and saved for later requests. Consequently, the first request may be substantially slower.

C.2 Row Serialization before Embedding

FAISS does not embed the raw CSV row object directly. Each source row is first converted into a LangChain `Document` containing two separate parts:

- `page_content`, the textual representation passed to the embedding model; and
- `metadata`, the structured information retained alongside the vector but not included in the embedding unless it is also written explicitly in `page_content`.

The repository contains two row-serialization workflows. The generic lazy index builder in `FaissIndexManager.row_to_text` produces the compact one-line representation shown above. It prepends the table name, then appends every non-empty value as `column: value`, separated by vertical bars. It excludes `__rid__` and all columns whose names end in `_rownum`. The associated metadata contains only the table name and row identifier:

```
page_content:
table: nation | n_nationkey: 1 | n_name: ARGENTINA | n_regionkey: 1

metadata:
{"table": "nation", "rid": "nation_2"}
```

The prebuilt REL-F1 BGE-M3 index uses the richer schema-aware workflow in `build_row_index.py`. Its `build_page_content` function creates a short natural-language row document with the following sections:

1. table identity and a statement that the document represents one row;
2. primary-key columns and values, when a key candidate is available;
3. up to 40 non-provenance column values, written with fully qualified names;
4. outgoing foreign-key relations selected from the schema profile; and
5. when the referenced row is found, up to four readable values from that row.

For example, a row from the `circuits` table is serialized as:

```
Row from table circuits.
This row represents one record from circuits.

Primary key:
circuits.circuitId = 1.

Column values:
circuits.circuitId = 1.
circuits.circuitRef = albert_park.
circuits.name = Albert Park Grand Prix Circuit.
circuits.location = Melbourne.
circuits.country = Australia.
circuits.lat = -37.8497.
circuits.lng = 144.968.
circuits.alt = 10.0.
```

Provenance columns matching `*_rownum` are not written into this text. Null values are represented as `NULL`; individual textual values are trimmed and truncated after 120 characters. Foreign-key descriptions have the form `source.column = value references target.column = value`. If the target row can be resolved, the serializer adds human-readable target attributes,

preferring columns whose names contain terms such as `name`, `title`, `status`, `type`, `region`, `country`, or `date`.

The rich document metadata stores `doc_type`, table, generated row ID, row-number column and value, primary-key values, the complete non-provenance row, and structured linked-row information. Before building FAISS, `build_row_faiss_index.py` normalizes this metadata by setting `rid` to the provenance row-number value (or, if unavailable, the generated row ID). This is the identifier expected by the gateway when it resolves a search result back to the exact CSV row.

The resulting process is therefore:

$$\text{CSV row} \longrightarrow \text{textual } \texttt{page\_content} \longrightarrow \text{BGE-M3 dense vector} \longrightarrow \text{FAISS},$$

while the metadata follows the vector into the index and is used only after retrieval. FAISS similarity search compares the embedded query with the embedded `page_content`; it does not compare the query directly with the metadata or the original CSV object.

C.3 Embedding Models and Strategies

The embedding backend is selected independently for each dataset. The TPC-H configuration uses `sentence-transformers/all-mpnet-base-v2` by default, through the Sentence Transformers backend. The REL-F1 row index uses **BGE-M3**, configured as `BAAI/bge-m3` with the `bge-m3` strategy. BGE-M3 is loaded through `FlagEmbedding.BGEM3FlagModel`.

Although BGE-M3 supports multiple retrieval representations, the current gateway uses only the model's dense vectors (`dense_vecs`) for both document and query embeddings before FAISS similarity search. Documents are encoded in batches with a maximum input length of 8192 tokens; queries are encoded individually with the same maximum length.

The `EmbeddingStrategies` wrapper supports three backends:

- `sentence-transformer`, implemented with `SentenceTransformer`;
- `bge-v1.5`, implemented with `FlagEmbedding.FlagModel`; and
- `bge-m3`, implemented with `FlagEmbedding.BGEM3FlagModel`.

When `EMB_STRATEGY=auto`, the wrapper infers the backend from the model name: `BAAI/bge-m3` selects BGE-M3, other `BAAI/bge-*` models select the BGE v1.5 path, and other names use Sentence Transformers. The shared `EMB_DEVICE` setting selects CUDA when available or CPU otherwise. BGE models use half precision on CUDA and full precision on CPU.

The model and strategy used to build an index must also be used when loading and querying it. Changing either setting requires rebuilding the corresponding FAISS index; otherwise, vector dimensions or embedding semantics may be incompatible.

C.4 Retrieval Modes

The standard RAG path performs one similarity search with `RETRIEVER_K` candidates and stops after `MAX_CONTEXT_ROWS` accepted rows. It limits the context to `MAX_TABLES` distinct tables.

Planner leaf paths use iterative retrieval. The first search requests `RETRIEVER_K` candidates; each later iteration increases k by the same amount. Retrieval stops after `MAX_ITERATIVE_RETRIEVALS` or when an iteration adds no new resolvable rows. Duplicate identifiers and rows that cannot be mapped back to the selected dataset are ignored.

When enabled, linked-row metadata may add correlated records up to `MAX_CORRELATED_CONTEXT_ROWS`. Dense retrieval is candidate selection, not proof of relevance. Exact row validation prevents a model from fabricating row contents, but it does not guarantee that all rows required for a query were retrieved.

D Leaf Output Contract and Validation

A leaf model must return a JSON array containing objects with exactly two keys:

```
[
  {
    "row_id": "nation_2",
    "values": {
      "nation_rownum": "nation_2",
      "n_nationkey": "1",
      "n_name": "ARGENTINA",
      "n_regionkey": "1",
      "__rid__": "nation_2"
    }
  }
]
```

Validation checks that:

- the root is an array;
- every item is an object with exactly `row_id` and `values`;
- `row_id` is a string present in the retrieved target-table context;
- valid row identifiers are not duplicated;
- `values` exactly equals the corresponding source row; and
- `values.__rid__`, when present, equals `row_id`.

The model is called at most twice. On failure, the second prompt emphasizes the JSON-only contract. The partial parser retains individually valid rows even if the complete output is invalid, enabling evaluation and downstream inspection without silently treating malformed items as trusted data.

E API

E.1 OpenAI-Compatible Endpoints

E.1.1 `GET /v1/models`

This endpoint returns logical identifiers rather than provider-specific model tags:

```
base-llama3-8b
best-ft-llama3-8b-nl
best-ft-llama3-8b-sql
planner-only
planner-only-pushdown
planner-only-explanation
internal-knowledge
iterative-join-aware
```

E.1.2 `POST /v1/chat/completions`

Accepted fields include `model`, `messages`, `temperature`, `stream`, and `dataset`. Extra fields are accepted for client compatibility. The gateway uses only the latest message whose role is `user`.

```
curl -X POST http://localhost:9006/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{
    "model": "planner-only",
    "dataset": "tpch",
    "messages": [{"role": "user", "content": "SELECT n_name FROM nation"}],
    "temperature": 0
  }'
```

The response follows the non-streaming OpenAI chat-completion envelope. Token counts are placeholders and return zero. Although the schema accepts `stream`, this endpoint does not emit OpenAI streaming chunks.

E.2 Provenance Explanation Endpoint

`POST /v1/provenance/explain` accepts a natural-language question and a previously generated answer JSON string. It parses the result/provenance array, resolves identifiers, detects structural problems, renders provenance expressions, and requests a concise domain explanation when all identifiers resolve. If any identifier is missing, model-based explanation is skipped and validation errors are returned.

E.3 Browser UI Endpoints

Endpoint	Purpose
`GET /`	Redirect to `/ui`.
`GET /ui`	Serve the interactive workspace.
`POST /ui/plan`	Build and return a SQL plan.
`POST /ui/run`	Run selected per-leaf pipelines and AP explanation synchronously.
`POST /ui/run/stream`	Emit NDJSON progress events and a final result.
`GET /ui/csv`	Inspect generated CSV files in the shared bucket.
`GET /debug/ui`	Inspect the latest in-memory chat and provenance-debug state.

The streaming UI endpoint uses NDJSON (`application/x-ndjson`), not Server-Sent Events and not the OpenAI streaming protocol.

F Model Routing and Providers

Logical identifiers keep clients stable while environment variables select concrete models. Base and fine-tuned modes normally route to Ollama. Planner leaf calls use `PLANNER_LLM_PROVIDER`, which may be `ollama`, `openai`, or `openai-compatible`. A compatible remote provider uses `LLM_API_BASE`, `LLM_API_KEY`, `LLM_API_MODEL`, request timeout, and TLS verification settings.

Temperature defaults to 0.0. Authentication headers received by the gateway are not used to protect these endpoints; deployment-level access control is required outside a trusted local environment.

G Configuration Reference

Area	Variables
Dataset	DEFAULT_DATASET, CSV_DIR, RELF1_CSV_DIR, RELF1_SCHEMA_PROFILE.
FAISS	FAISS_INDEX_FOLDER, RELF1_FAISS_INDEX_FOLDER, INDEX_TABLES, RELF1_INDEX_TABLES, INDEX_BATCH_SIZE.
Embeddings	EMB_MODEL, EMB_STRATEGY, EMB_DEVICE, RELF1_EMB_MODEL, RELF1_EMB_STRATEGY.
Retrieval	RETRIEVER_K, MAX_ITERATIVE_RETRIEVALS, MAX_CONTEXT_ROWS, MAX_TABLES, INCLUDE_CORRELATED_CONTEXT_ROWS, MAX_CORRELATED_CONTEXT_ROWS.
Ollama	OLLAMA_HOST, OLLAMA_NUM_CTX, OLLAMA_REQUEST_TIMEOUT, and model-specific OLLAMA_MODEL_* variables.
Compatible LLM	PLANNER_LLM_PROVIDER, PLANNER_LLM_MODEL, LLM_API_BASE, LLM_API_KEY, LLM_API_MODEL, LLM_SSL_VERIFY, LLM_REQUEST_TIMEOUT.
Explanation	EXPLANATION_URL, EXPLANATION_ENDPOINT, EXPLANATION_BUCKET_DIR, EXPLANATION_REQUEST_TIMEOUT, EXPLANATION_CSV_DELIMITER, EXPLANATION_KEEP_ROWNUM.
Rendering and logging	GATEWAY_RESPONSE_FORMAT, GATEWAY_MARKDOWN_INCLUDE_RAW, GATEWAY_MARKDOWN_MAX_ROWS, GATEWAY_LOG_PATH.

H Deployment and Operation

H.1 Prerequisites

- Docker with Compose support;
- Git and Git LFS for the fine-tuned model artifacts;
- sufficient disk and memory for model serving and embedding indexes; and
- network access to any configured remote LLM endpoint.

H.2 Starting the Stack

```
git lfs pull
docker compose up -d --build
docker compose ps
```

The current Compose mapping exposes the gateway on `127.0.0.1:9006`, the explanation service on host port `5002`, and keeps Ollama internal to the Compose network. The `ollama-init` service waits for Ollama, pulls the base model if needed, and creates fine-tuned model tags from their `Modelfile` definitions.

```
docker compose logs -f gateway
```

H.3 Persistence

- Ollama model data is stored in the `ollama_data` volume.
- PostgreSQL state is stored in `wp4_poc_postgres_data`.
- Indexes, datasets, logs, UI files, and the shared CSV bucket are bind-mounted from the repository.

I Observability and Error Behavior

The gateway writes JSON Lines events to `GATEWAY_LOG_PATH`. Events include request metadata, selected dataset and model, retrieval iterations, row identifiers, contexts, planner results, prompts, raw model outputs, validation status, generated files, explanation output, and errors.

`/debug/ui` stores only the latest debug state in process memory. It is overwritten by later requests, is not durable, and may expose source data and prompts. Logs and debug endpoints should be treated as sensitive outside a demo.

Explicitly handled client errors return HTTP 400. Pipeline and provider failures generally return HTTP 500. The UI streaming endpoint instead emits structured `error` or `fatal_error` events so the browser can show partial progress.

References

[1] Peter Buneman, Sanjeev Khanna, and Tan Wang-Chiew. Why and where: A characterization of data provenance. In *International conference on database theory*, pages 316–330. Springer, 2001.

[2] James Cheney, Laura Chiticariu, Wang-Chiew Tan, et al. Provenance in databases: Why, how, and where. *Foundations and Trends® in Databases*, 1(4):379–474, 2009.

[3] Boris Glavic. Data provenance: Origins, applications, algorithms, and models. *Foundations and Trends in Databases*, 9(3–4):1–232, 2021. Illinois Institute of Technology.

[4] Todd J Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pages 31–40, 2007.

[5] Dario Satriani, Enzo Veltri, Donatello Santoro, Sara Rosato, Simone Varriale, and Paolo Papotti. Logical and physical optimizations for sql query execution over large language models. In ACM, editor, *SIGMOD/PODS 2025, ACM International Conference on Management of Data, June 22-27, 2025, Berlin, Germany / Also published in Proceedings of the ACM on Management of Data, Vol. 3, Issue 3, Article N°181*, Berlin, 2025. © ACM, 2025. This is the author's version of the work. It is posted here by permission of ACM for your personal use. Not for redistribution. The definitive version was published in SIGMOD/PODS 2025, ACM International Conference on Management of Data, June 22-27, 2025, Berlin, Germany / Also published in Proceedings of the ACM on Management of Data, Vol. 3, Issue 3, Article N°181 https://doi.org/10.1145/3725411.

[6] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. RAT-SQL: relation-aware schema encoding and linking for text-to-sql parsers. *CoRR*, abs/1911.04942, 2019.

[7] Xiaohan Yu, Pu Jian, and Chong Chen. Tablerag: A retrieval augmented generation framework for heterogeneous document reasoning, 2025.